

SO SÁNH ABSTRACT CLASS VÀ INTERFACE TRONG JAVA

1. Giới thiệu chung

Trong Java, cả **Lớp trừu tượng (Abstract Class)** và **Giao diện (Interface)** đều là công cụ để thực hiện tính Trừu tượng (Abstraction). Chúng đều có thể chứa các phương thức trừu tượng (phương thức chưa có phần thân) và đều không thể khởi tạo đối tượng trực tiếp.

Tuy nhiên, sự tồn tại song song của hai khái niệm này không phải là ngẫu nhiên. Chúng phục vụ cho các mục đích thiết kế kiến trúc phần mềm hoàn toàn khác nhau. Việc lựa chọn sai giữa hai khái niệm này có thể dẫn đến một hệ thống khó bảo trì và mở rộng.

2. So sánh chi tiết

Để phân biệt rõ ràng, chúng ta sẽ xem xét trên 5 tiêu chí kỹ thuật cốt lõi:

2.1. Về Bản chất và Quan hệ

- **Abstract Class:** Đại diện cho mối quan hệ **"IS-A"** (Là một). Nó được dùng để xây dựng một lớp cha cơ sở cho các lớp con có cùng bản chất.
 - Ví dụ: **Cho** là một **DongVat** (Abstract Class).
- **Interface:** Đại diện cho mối quan hệ **"CAN-DO"** (Có thể làm gì/Có khả năng gì) hoặc **"HAS-A"** (Có hành vi). Nó đóng vai trò là một bản hợp đồng về hành vi.
 - Ví dụ: **Cho** có khả năng **Chay** (Interface).

2.2. Về Phương thức (Methods)

- **Abstract Class:** Rất linh hoạt. Có thể chứa phương thức trừu tượng (abstract), phương thức thông thường (concrete), và hỗ trợ mọi phạm vi truy cập (**public**, **protected**, **private**).
- **Interface:**
 - Mặc định các phương thức là **public abstract**.
 - Từ Java 8: Hỗ trợ thêm phương thức **default** và **static** (có phần thân).
 - Từ Java 9: Hỗ trợ thêm phương thức **private**.
 - Tuy nhiên, Interface không thể có phương thức **protected**.

2.3. Về Biến và Hằng số (Variables)

- **Abstract Class:** Có thể chứa các biến instance (non-static), biến static, hằng số (**final**) hoặc biến thông thường (non-final). Nó có khả năng **lưu trữ trạng thái** của đối tượng.
- **Interface:** Chỉ có thể chứa hằng số. Mọi biến khai báo đều ngầm định là **public static final**. Interface **không lưu trữ trạng thái**.

2.4. Về Kế thừa (Inheritance)

- **Abstract Class:** Java chỉ hỗ trợ đơn kế thừa với Class. Một lớp con chỉ có thể **extends** duy nhất một Abstract Class.
- **Interface:** Java hỗ trợ đa kế thừa với Interface. Một lớp có thể **implements** (thực thi) nhiều Interface cùng lúc.

2.5. Về Hàm khởi tạo (Constructor)

- **Abstract Class:** Có Constructor. Constructor này được dùng để khởi tạo các thuộc tính của lớp cha khi lớp con được tạo ra.
- **Interface:** Không có Constructor. Vì Interface không có trạng thái (instance variables) để khởi tạo, nên Constructor là không cần thiết.

3. Bảng tổng hợp so sánh

Dưới đây là bảng tóm tắt giúp sinh viên dễ dàng ôn tập:

Tiêu chí	Abstract Class	Interface
Từ khóa	abstract class, extends	interface, implements
Đa kế thừa	Không (Chỉ mở rộng 1 lớp)	Có (Thực thi nhiều giao diện)
Trạng thái	Có thể lưu trạng thái (biến instance)	Không lưu trạng thái (chỉ hằng số)
Constructor	Có	Không
Tốc độ	Nhanh hơn (liên kết trực tiếp)	Chậm hơn đôi chút (cần tìm phương thức)
Phạm vi truy cập	public, protected, private...	Mặc định là public (Java 9+ có private)
Mục đích chính	Chia sẻ code chung, quan hệ huyết thống.	Định nghĩa chuẩn hành vi, đa kế thừa.

4. Hướng dẫn lựa chọn (Best Practices)

Trong thực tế thiết kế, hãy dựa vào các gợi ý sau để quyết định sử dụng loại nào:

Nên sử dụng Abstract Class khi:

1. Bạn muốn chia sẻ mã nguồn (code reusability) giữa các lớp có quan hệ chặt chẽ.
2. Bạn cần định nghĩa các biến instance (không phải hằng số) để các lớp con có thể thay đổi giá trị.
3. Bạn muốn sử dụng các phạm vi truy cập như `protected` hoặc `private` cho phương thức.

Nên sử dụng Interface khi:

1. Bạn muốn các lớp không liên quan đến nhau (không cùng họ hàng) thực hiện cùng một hành vi (Ví dụ: Lớp `NhanVien` và lớp `MayMoc` đều có thể `LamViec`).
2. Bạn muốn định nghĩa một API chuẩn mà không quan tâm ai sẽ thực thi nó.
3. Bạn cần tận dụng tính năng đa kế thừa của Java.

5. Kết luận

Abstract Class và Interface không phải là hai khái niệm đối nghịch mà là hai công cụ bổ trợ cho nhau.

- **Abstract Class** xây dựng nên bộ khung xương sống (Skeleton) cho hệ thống phân cấp.
- **Interface** đóng vai trò như các mảnh ghép chức năng (Plug-ins) giúp hệ thống linh hoạt và dễ mở rộng.

Một kiến trúc phần mềm tốt thường kết hợp cả hai: Sử dụng Interface để định nghĩa các dịch vụ (Service) và sử dụng Abstract Class để cung cấp các cài đặt cơ sở mặc định.